Integrated Optimization of Product-Service System Configuration and Delivery with Differentiated Valid Cut Bundle Selection across Various Branch-and-Bound Solving Stages

Haoyang Cao+
Institute of Software
University of Chinese
Academy of Sciences
Beijing, China
haoyangc2001@gmail.com

Jiakang Chen+
School of Mechanical Engineering
Shanghai Jiao Tong Universityn
Shanghai, China
jkchen810975@gmail.com
+ Equal contribution

Shenhao Lou
Weiyang College
Tsinghua University
Beijing, China
loush22@mails.tsinghua.edu.cn

Yilun Zhang*
Department of Industrial Engineering and Management
Shanghai Jiao Tong University
Shanghai, China
asdf164@126.com
*Corresponding author

***Abstract*—To address the correlation between Product-Service System (PSS) configuration and delivery scheduling, and overcome inefficiencies of existing valid cut selection, this study proposes an integrated optimization framework. We first formulate the problem as a Mixed Integer Linear Programming (MILP) model, balancing customer utility, service timeliness and enterprise profit with constraints on inventory, delivery capacity and route continuity. A key innovation is a machine learning-based differentiated valid cut bundle selection strategy: strong cuts for branch-and-bound root nodes to reduce branching burden, and lightweight cuts for non-root nodes to boost efficiency. Regression models replace classification algorithms to predict cut bundle computation time, providing quantitative decision support. Numerical experiments with 4,928 cases validate the approach. Gradient Boosting Decision Tree achieves 99.5% prediction accuracy, and the differentiated strategy outperforms single-stage or uniform cut application in computational efficiency. This work enhances the efficiency of PSS joint optimization, offering a practical solution for enterprises.**

Keywords—Product-Service System, Mixed Integer Linear Programming, Valid Cuts, Machine Learning

I. INTRODUCTION

The enterprise needs to select proper Product-Service System (PSS) that integrates physical products with value-added services [1, 2] to meet the diverse customer needs and improve overall customer satisfaction [3, 4]. This is the so-called PSS configuration. Several challenges may arise during PSS configuration and the delivery procedure to corresponding customer. For instance, selecting high-quality services in the configuration phase may be time-consuming, making it difficult to ensure the delivery efficiency. It should be noted that customer satisfaction with a PSS configuration plan depends not only on its alignment with customer-specific needs and preferences, but also on the efficiency of the delivery sequence. Moreover, designing delivery plans requires the consideration of service delivery efficiency, which is influenced by both the service selection and the route planning of the service delivery procedure. Therefore, the configuration and delivery decisions are inherently correlated. From these perspectives, it is essential to integrate the configuration and delivery planning, taking into account the interests of all stakeholders [5, 6]. The variability in delivery costs and time windows, and the heterogeneous service completion time of different service levels further complicate the configuration and delivery decisions.

To cope with these challenges, previous studies employ exact algorithms to solve configuration and delivery planning problems with the aim of maximizing customer satisfaction and service delivery efficiency. Some studies augment computation efficiency by introducing valid cuts [7, 8], and customizing these cuts based on specific problem scenarios and solution space characteristics [9, 10]. Some other works investigate the underlying acceleration mechanisms of valid cuts, revealing that the optimal set bundle typically includes only a subset of all available cuts rather than the full set [11], thus highlighting the necessity of self-adaptive cut bundle selection. To facilitate this process, machine learning (ML) techniques are applied to self-adaptively recommend valid cut bundles under different cases [6, 12-14]. Although such methods can identify appropriate cut bundles, they still exhibit several limitations, which motivate the two key contributions proposed in this study.

Firstly, previous studies typically apply static valid cut bundles throughout the entire solving process, neglecting the various requirements across different solving stages. To be more specific, different stages of a branch-and-bound algorithm may benefit from differentiated strategies for valid cut selection. For example, at the root node, it is generally preferable to adopt strong cuts that significantly tighten the relaxation, in order to reduce the burden of subsequent

branching. At this stage, more cuts and longer computation time are relatively acceptable. In contrast, non-root nodes favor significant improvements in computation efficiency accompanied by parallel computation, thus requiring fewer and simpler cuts. Recognizing these stage-specific requirements, our study adopts ML to differentially select valid cut bundles across two solving stages of the branch-and-bound algorithm.

Secondly, prior research primarily utilizes classification ML algorithms, offering categorical predictions for the optimal cut bundle (e.g., the ID of the best cut bundle, or the rank of one certain cut bundle). However, such classification outcomes fail to display the details of the efficiency gap among different bundles. For instance, if the classification algorithm predicts that cut bundle A outperforms B, this may correspond to two circumstances respectively. First, a significant time difference (e.g., A: 10s, B: 20s) clearly shows A's superiority, allowing the ML algorithm to easily identify the superior cut bundle. Second, minor differences (e.g., A is 1s faster) may stem from the operating system's resource allocation rather than the cuts' effectiveness. Actually, bundle B may exhibit a better acceleration effect. In this case, the confidence of prediction results will be diminished, since solely relying on the classification results cannot distinguish the above two circumstances effectively. Consequently, the decision-makers may suffer from the risk from probably unreliable information yielded by the classification algorithm. And it could be challenging for the users to judge the confidence of the predicted conclusion that whether one cut bundle dominates another. Therefore, only adopting classification results could not be sufficiently informative for valid cut selection. To overcome this issue, it is advisable to acquire the specific computation time by employing a regression algorithm instead of the classification one, as such additional information offers the users a clearer sense of the difference of computation efficiency. It opens up new opportunities for decision makers to assess to what extent the forecast results can be trusted more accurately. In addition, some users prefer to incorporate their own preferences and expertise into the decision-making process, not completely following the prediction outputs which are viewed as a recommendation. Therefore, they need the computation time forecast as supplementary quantitative details to reinforce subjective decisions on cut bundle selection. In conclusion, conducting a regression task is necessary to better fix the aforementioned problems.

The remainder of the paper is organized as follows. Section II quantifies our problem in a mathematical formulation. Section III describes the prediction algorithm to differentially select valid cut bundles. Section IV reports numerical results to validate our academic contributions. Section V summarizes the research and provides extensions for future work.

II. Problem Formulation

We introduce basic settings of our research problem, including parameters, decision variables, objectives and constraints, before formulating it as an integer programming.

A. Problem Description

A PSS is deployed by enterprises to meet heterogeneous customer demands through multiple services with supplementary support products. As an example, Computer Numerical Control (CNC) machine suppliers provide not only basic health diagnostic services but also more advanced diagnostic options. While these advanced services offer enhanced performance, they require longer delivery time, higher costs, and rely on scarce and specialized resources. The enterprises will collaborate with customers to customize their product and service configurations. Engineering teams will deliver corresponding services, encompassing on-site product delivery, installation, and operational training. During the configuration process of a PSS, it is crucial to take the delivery procedure into account, and vice versa. For example, the advanced health diagnostic PSS enhances performance versus its basic counterpart but extends delivery time significantly. Configuration based solely on functionality favors the advanced service; when considering both functionality and delivery time, the selection may shift to the basic service.

Given these considerations, we adopt a two-phase joint optimization approach that simultaneously addresses the configuration of PSS and order delivery scheduling. In the first stage, appropriate services and their corresponding products are selected for customers based on individual needs. In the second stage, the delivery sequence for multiple customers is designed for each engineering team to improve efficiency and reduce costs. During the configuration phase, enterprises should evaluate to what extent different PSS bundles can meet customer functional requirements and consider key factors affecting the delivery efficiency, such as service completion time and delivery sequences. Additionally, the feasibility and cost-effectiveness of the delivery sequence can also shape the configuration of a PSS.

B. Variables and Parameters

To construct the optimization model, the following foundational assumptions are defined:

1) The production strategies of products can meet the needs of service delivery;

2) Production capacity is sufficient, and production time is negligible compared to service time, such that the system can be approximated as a pure service system.

Afterwards, we will characterize the decision variables and input parameters quantitatively and define the objectives and the constraints.

Taking the PSS of the CNC machine tool fault detection as an example, we introduce the joint optimization process of product service configuration and order delivery scheduling, which can be generalized to any PSS. An enterprise provides different levels of maintenance training services for each customer i (i=1, 2, ..., I) in network GN. We characterize the service in different levels, denoted as m (m=1, 2, ..., M), e.g. m=1 represents the advanced service and m=2 represents the basic service. Each level of service requires several products for functional support (e.g. mechanical parts used for maintenance), forming a product-based PSS. Each service (representing one PSS) spends S_m units of time, charging a price of P_m (containing the price of the service and supporting products), accompanied by a total cost of C_m (comprising the service operational cost, the production cost, and the holding

cost of the corresponding products). Generally, selecting advanced services for customers means bundling advanced products, which also have higher prices. Each product n (n=1, 2, ..., N) can support multiple services, and its available inventory is denoted by A_n . We use Q_{mn} to denote the correspondence between products and services, i.e., service m requires Q_{mn} product n. Due to the diversified capability of the aforementioned products and services in meeting customers' functional requirements, we use U_{im} to quantify the utility of offering the PSS associated with service m to customer i. Note that U_{im} encompasses not only the utility provided by the service itself, but also the utility of correlated products.

Each customer may impose restrictions on the earliest and latest service starting time of the engineering team, which are denoted as G_i and H_i respectively, due to customers' own preferences. To meet the needs of different customers, multiple engineering teams, indexed by k, k=1, 2, ..., K, should jointly accomplish the delivery. Each team departs from and returns to the enterprise, denoted as node 0 and node I+1 in the network GN respectively. The other nodes represent the locations of each customer. Let T_{ij} be the transition time from customer i to j. Costs are incurred when engineering teams visit customers, including a fixed cost F (related to whether to attend PSS delivery), and a variable cost D_{ij} (affected by the distance, such as mileage-driven expenses). The delivery capability of each engineering team is restricted by E, which represents the total time limit.

Based on the above settings, the relevant decision variables are given. r_{im} is a binary variable indicating whether to assign service m to customer i. r_{im} equals 1 if the service is assigned. This variable characterizes the first stage of the decision. We then characterize the service delivery sequence, denoted as a binary variable x_{ijk} . If team k visits customers i and j consecutively, then x_{ijk} equals 1, which characterizes the core decision of the second stage. Also, we can derive whether team k visits customer i (denoted as a binary variable y_{ik}) and whether team k attends delivery tasks (denoted as a binary variable z_k). In addition, we introduce an integer variable t_{ik} to record the time at which team k arrives customer i and starts the service.

C. Objectives and Constraints

This section presents an integer programming model that integrates the objectives of customers and enterprises. Customers aim to maximize the utility-to-price ratio and minimize service start time, as formulated in Formula (1), while enterprises seek to maximize profit, calculated by the revenue from PSS delivery subtracted by the fixed and variable costs during the delivery process, as shown in Formula (2).

$$\max \sum_{i=1}^{I}\sum_{m=1}^{M} r_{im}\frac{U_{im}}{P_m} - \sum_{i=1}^{I}\sum_{k=1}^{K} t_{ik} \tag{1}$$

$$\max \sum_{i=1}^{I}\sum_{m=1}^{M} r_{im}(P_m - C_m) - \sum_{k=1}^{K} z_k F - \sum_{i=0}^{I+1}\sum_{j=0}^{I+1}\sum_{k=1}^{K} x_{ijk} D_{ij} \tag{2}$$

Conclusively, the weighted total objective consisting of Formula (1) and (2) is:

$$\max\ \alpha_1 \sum_{i=1}^{I}\sum_{m=1}^{M} r_{im}\frac{U_{im}}{P_m} - \alpha_2 \sum_{i=1}^{I}\sum_{k=1}^{K} t_{ik} + \alpha_3\Big(\sum_{i=1}^{I}\sum_{m=1}^{M} r_{im}(P_m - C_m) - \sum_{k=1}^{K} z_k F - \sum_{i=0}^{I+1}\sum_{j=0}^{I+1}\sum_{k=1}^{K} x_{ijk} D_{ij}\Big)$$

where $\alpha_1, \alpha_2, \alpha_3$ are the coefficients for each objective term, which can also be used for normalization.

Subsequently, we propose the following constraints that encompass both the configuration and delivery phases.

(1) Service configuration requirements

As shown in Constraint (3), for each customer, at most one service can be configured since services only differ in levels. In certain cases, no service can be configured, which means that due to inventory or delivery capacity constraints, the enterprise may refuse to offer any service to one customer.

$$\sum_{m=1}^{M} r_{im} \le 1 \qquad \forall i \in \{1, ..., I\} \tag{3}$$

Particularly, as shown in Constraint (4), the product service configuration is not permitted at the origin node or the terminal node.

$$\sum_{m=1}^{M} (r_{0m} + r_{(I+1)m}) = 0 \tag{4}$$

(2) Product inventory limitation

The on-hand inventory of each product is limited within a specific period, which represents the constraint of production capacity. Constraint (5) represents that the available quantity of each product for customers in configuration cannot exceed its total amount A_n.

$$\sum_{i=1}^{I}\sum_{m=1}^{M} r_{im} Q_{mn} \le A_n \qquad \forall n \in \{1, ..., N\} \tag{5}$$

(3) Delivery capacity limitation

As shown in the Constraint (6), all engineering teams have a total working time limit, so they must accomplish the assigned delivery tasks within the time limit. Note that if an engineering team doesn't attend the service delivery, then $t_{(I+1)k}$ is 0, and the values of the other variables t_{ik} hold no practical meaning.

$$t_{(I+1)k} \le z_k E \qquad \forall k \in \{1, ..., K\} \tag{6}$$

(4) Flow balance constraints

$$\sum_{j=0}^{I+1} x_{0jk} - \sum_{j=0}^{I+1} x_{j0k} = z_k \qquad \forall k \in \{1, ..., K\} \tag{7}$$

$$\sum_{j=0}^{I+1} x_{j(I+1)k} - \sum_{j=0}^{I+1} x_{(I+1)jk} = z_k \qquad \forall k \in \{1, ..., K\} \tag{8}$$

$$\sum_{i=0}^{I+1} x_{ijk} - \sum_{i=0}^{I+1} x_{jik} = 0 \qquad \forall k \in \{1, ..., K\},\ j \in \{1, ..., I\} \tag{9}$$

The delivery problem is essentially a Vehicle Routing Problem (VRP), thus the classical flow balance constraints can be applied to characterize origin, middle, and terminal nodes. Constraint (7) ensures that each team attending the delivery tasks starts at the enterprise; Constraint (8) ensures that each

team ultimately returns to the enterprise after completing the tasks; Constraint (9) guarantees that if one team reaches one customer node, it must leave from that node.

(5) Route continuity constraints

When accessing customer i followed by customer j, the service starting time for two consecutive customers must satisfy Constraint (10):

$$t_{ik} + T_{ij} + \sum_{m=1}^{M} r_{im} S_m - t_{jk} \le BigM(1 - x_{ijk}) \quad \forall i,j \in \{0,...,I+1\}, k \in \{1,...,K\} \tag{10}$$

in which $BigM$ denotes a sufficiently large positive number.

Note that Constraint (10) can also eliminate subtours, such as the impossibility of returning to customer i after leaving i.

(6) Service starting time window constraints

As shown in Constraint (11), each customer is associated with a specific starting time window $[G_i, H_i]$. Therefore, for each engineering team, the service must be scheduled to ensure that the starting time lies within the designated time window for assigned customers.

$$G_i - (1 - y_{ik}) BigM \le t_{ik} \le H_i + (1 - y_{ik}) BigM \quad \forall i \in \{1,...,I\},\ k \in \{1,...,K\} \tag{11}$$

(7) Variable rationality constraints

If a certain service with related products is allocated to one customer, an engineering team will be assigned to accomplish the delivery by visiting only once. If no PSS is configured for a customer, then no engineering team will visit this customer, as shown in Constraint (12). Note that variables y_{ik} corresponding to the origin node and the terminal node are all set to 0.

$$\sum_{k=1}^{K} y_{ik} = \sum_{m=1}^{M} r_{im} \qquad \forall i \in \{0,...,I+1\} \tag{12}$$

Constraint (13) states that self-transition, i.e., transferring from node i to itself, is prohibited.

$$\sum_{i=0}^{I+1} \sum_{k=1}^{K} x_{iik} = 0 \tag{13}$$

Constraint (14) limits a direct transition from the origin to the terminal, or from the terminal to the origin.

$$\sum_{k=1}^{K} (x_{0(I+1)k} + x_{(I+1)0k}) = 0 \tag{14}$$

Constraint(15) states that an engineering team must deliver the corresponding service when visiting customer i, and vice versa.

$$y_{ik} = \sum_{j=0}^{I+1} x_{jik} \qquad \forall i \in \{1,...,I\}, k \in \{1,...,K\} \tag{15}$$

As shown in Constraints (16) and (17), when an engineering team works, it will serve at least one customer; otherwise, it doesn't attend service delivery, in which case all corresponding variables y_{ik} are set to 0. Additionally, variables $t_{ik}$are set to 0 for the unvisited location.

$$z_k \ge \frac{\sum_{i=1}^{I} y_{ik}}{BigM} \qquad \forall k \in \{1,...,K\} \tag{16}$$

$$z_k \le BigM \sum_{i=1}^{I} y_{ik} \qquad \forall k \in \{1,...,K\} \tag{17}$$

In particular, the service starting time at enterprises for all engineering teams are commonly forced to 0. Constraint (18) corresponds to this regulated setting.

$$\sum_{k=1}^{K} t_{0k} = 0 \tag{18}$$

Conclusively, we can formulate the PSS configuration and delivery problem as a Mixed Integer Linear Programming (MILP) model. To solve the MILP model, we adopt the branch-and-bound framework with a series of valid cuts to tighten the relaxation bound and accelerate the branch-and-bound procedure. Furthermore, we proposed a prediction algorithm based on the ML, which aims to consider characteristics of specific cases, cover different solving stages of the branch-and-bound algorithm, and automatically recommend the most suitable valid cut bundle.

III. Algorithm Design

The branch-and-bound algorithm employs a linear relaxation of the original problem, branching on non-integer variables [9]. This process yields a relaxation bound from the linear relaxation and a feasible bound reflecting the best-known objective value. Convergence to the global optimum is determined by the difference between these bounds. To further enhance the performance of the branch-and-bound algorithm, introducing valid cuts is an effective approach. These cuts tighten the relaxation bound without shaping the feasible bound and boost the convergence of the branch-and-bound algorithm. They eliminate some solutions feasible in the linear relaxation but infeasible in the original integer programming, thereby pruning the search space and reducing unnecessary explorations. With respect to the model characteristics, we devise a series of customized valid cuts and propose a machine learning-based selection strategy. The strategy differentially chooses the optimal valid cut bundle at root nodes and non-root nodes respectively. We then elaborate on this in the following.

A. Customization of Valid Cuts

Valid cuts can extract space structure information of the solution, thereby reducing unnecessary explorations and accelerating the convergence of the branch-and-bound algorithm. In the following sections, we propose a series of customized valid cuts for the aforementioned MILP model and justify their validity.

For any pair of customers i and j, each engineering team can only consecutively visit either from i to j, or from j to i, which has already been guaranteed by Constraint (10). Notably, Constraint (19) serves the same purpose and removes integer solutions that are already infeasible. Since both variables x_{ijk} and x_{jik} in the original problem are binary, adding Constraint (19) does not change the feasible region and optimal objective value. However, in the linearly relaxed problem, where x_{ijk} and x_{jik} are relaxed to continuous variables between 0 and 1, it is possible that $x_{ijk} + x_{jik} > 1$. With the aid of Constraint (19), this

case no longer exists, so the cut removes the feasible solution of the relaxed problem. According to the aforementioned definition of valid cuts, Constraint (19) can be considered as a valid cut.

$$x_{ijk} + x_{jik} \leq 1 \qquad \forall i,j \in \{1,...,I\}, k \in \{1,...,K\} \tag{19}$$

If an engineering team visits customers i and j consecutively, the latest service starting time at node j cannot be less than the sum of the service starting time at node i, the service completion time and the transition time from i to j, which can be guaranteed by Constraints (10) and (11) For the original integer programming model, Constraint (20) removes the impossible case that the aforementioned latest service starting time is earlier than the sum of the service starting time at node i, the service completion time and the transition time. However, this case may happen in the linearly relaxed problem without Constraint (20) . Constraint (20) removes such solutions that are feasible in the linear relaxation but infeasible in the integer programming problem. From this perspective, it can also be considered as a valid cut.

$$t_{ik} + \sum_{m=1}^{M} r_{im} S_m + T_{ij} - BigM(1 - x_{ijk}) \leq H_j \quad \forall i,j \in \{1,...,I\}, k \in \{1,...,K\} \tag{20}$$

Under the assumption that each customer is served immediately upon being visited, if a team visits customers i and j consecutively, the latest service starting time at node j cannot be earlier than the sum of the earliest service starting time at node i, the minimum service completion time at node i, and the transition time from i to j. Similar to the explanation of Constraint (20), Constraint (21) can also serve as a valid cut.

$$G_i + T_{ij} + \min(S_m) - BigM(1 - x_{ijk}) \leq H_j \quad \forall i,j \in \{1,...,I\}, k \in \{1,...,K\} \tag{21}$$

B. Differentiated Selection for Valid Cut Bundles

While valid cuts can accelerate the convergence of the branch-and-bound algorithm, they simultaneously introduce additional complexity constraints to subsequent subproblems, inevitably slowing down the solving process. When the computation deceleration induced by these supplementary constraints outweighs the convergence acceleration, it can hardly be viewed as an improvement in computation efficiency caused by adding more valid cuts. Therefore, when dealing with specific cases, optimal computation efficiency may be achieved by incorporating partially valid cuts instead of all cuts. In addition, during the pre-optimization process, it is found that the optimal valid cut bundle varies across cases, and the variation pattern is hard to accurately quantify, so it is difficult to select the most suitable valid cut bundle using a "white-box" method when facing new cases. Meanwhile, testing all possible cut bundles to identify the optimal one incurs high computation costs, which is impractical in real world applications with a large number of candidate cut bundles or strict time limits. Therefore, when facing any new case, we need to quickly select the most suitable valid cut bundle. However, how to select the most suitable valid cut bundle remains a challenge. Although previous studies utilize ML for cut bundle selection, most have focused on classic cutting planes for simple operations research problems. Our problem is more complex than classic ones, and the cuts to be selected are customized based on problem characteristics, with the structural differences from classic cutting planes. Also, traditional selection algorithms mainly rely on deep learning, whose training complexity and model interpretability often fail to meet the needs of practical application scenarios. Thus, traditional methods can hardly be applicable to our study, and our valid cut bundle selection method introduces a novel and more practical solution.

It should be emphasized that the requirements for valid cut bundles differ between the root node and the non-root nodes in the branch-and-bound tree. The root node critically influences subsequent search processes across all descendant nodes. Rather than prioritizing computation speed, it proves more advantageous to adopt valid cuts with stronger cutting capabilities to eliminate relaxation solutions, accepting manageable efficiency compromises to achieve tighter relaxation bounds. By allocating additional computation time at the root node with tighter valid cuts, the search space can be significantly reduced, thereby decreasing the workload required for subsequent branch-and-bound searches. Conversely, for the non-root nodes, the primary objective shifts to expedited solution completion. Under parallel computing, sufficient node exploration enables rapid convergence without requiring complex valid cut enhancements at individual nodes, avoiding excessive time consumption. However, most conventional studies exhibit insufficient investigation into the underlying mechanisms behind such differences and lack dedicated algorithmic strategies, a gap that this paper seeks to address. Specifically, the proposed valid cut bundle consists of two components corresponding to the root node and non-root nodes in the branch-and-bound tree. Recognizing the divergent requirements for valid cut bundles between these node types, our approach differentially selects cut bundles across various stages of the branch-and-bound algorithm to optimize global computation efficiency. Notably, this does not mean devising two separate selection algorithms based on identical principles. Given the inherent correlation between root and non-root nodes, our methodology adopts a single algorithmic architecture that simultaneously generates differentiated selection outcomes for valid cuts.

To develop a universal differentiated selection algorithm for valid cut bundles, we adopt "black-box" algorithms such as ML while designing customized feature sets calibrated to problem-specific characteristics. Building upon this framework, our approach predicts computation time corresponding to different valid cut bundles based on input parameters of specific cases. This enables rapid identification of the optimal valid cut bundle with superior acceleration performance without solving the MILP, thereby achieving case-specific differentiated selection.

We utilize supervised learning algorithms to achieve differentiated selection. Initially, based on a large number of randomly generated cases, mathematical models are constructed for all valid cut bundles involving two types of nodes: root nodes and non-root nodes. A commercial optimizer is then utilized to obtain the globally optimal solution for each mathematical model, during which input parameters and computation time under various valid cut bundles are recorded.

Then the raw data undergoes preprocessing to build a sample dataset, where the input parameters of the cases are transformed into features for the supervised learning algorithm, serving as the basis for selecting valid cuts (e.g., the time windows, the transition time, the service completion time, the delivery cost, etc.). The computation time of each valid cut bundle for root nodes and non-root nodes is used as the label for the supervised learning algorithm. On the above basis, Mean Square Error (MSE) is selected as the loss function, and R^2 is chosen as the evaluation metric for training the supervised learning algorithm [15]. Additionally, cross-validation and hyperparameter tuning are employed to further enhance prediction accuracy. Up to this point, the entire algorithmic procedure has been outlined. The subsequent sections will focus on further elaborating the feature design.

Features are crucial for valid cut bundle selection. Since the most suitable valid cut bundle varies across cases, the used features should be able to effectively capture the case information. To this end, grounded on the MILP input parameters, we pick some essential parameters which earns a satisfactory accuracy according to numerical experiments. Note that some optimization model input parameters cannot be directly used as features in supervised learning; otherwise, different samples would have various feature dimensions, which makes training procedures inapplicable. For instance, in Case 1 (5 nodes), distance parameters D_{ij} form a 25-dimensional feature, whereas in Case 2 (10 nodes) the dimension size expands to 100. This implies that adopting original values of MILP parameters may lead to inconsistent feature dimension. To address the issue, we employ some descriptive statistics (e.g. mean, standard deviation, etc.) to replace corresponding original values as features. For instance, replacing the original D_{ij} values with their mean and standard deviation, regulates the feature dimensions to 2. This feature engineering not only unifies the feature dimension across different training samples, but also extracts useful information from MILP parameters for valid cut bundle selection.

For *I, M, N,* and *K,* their original values are used as features. For the other parameters, several commonly used and effective descriptive statistics are employed. For x_{ijk}, its median, mode, and standard deviation are taken as features. For S_m, its minimum, maximum, mean, standard deviation, kurtosis, and quantiles serve as features. Regarding D_{ij}, the features incorporate minimum, maximum, mean, standard deviation, median, kurtosis, and quantiles. The parameters C_m, G_i, H_i derive features from minimum, maximum, mean, standard deviation, median, mode, and quantiles. For P_m, A_n, T_{ij}, their minimum, maximum, mean, standard deviation, median, mode, kurtosis, and quantiles are used as features. It should be noted that the above features have already been refined by feature selection operations in order to reduce the overfitting risk by removing partial mutually correlated features and improve the prediction accuracy, elaborated in Section 4.3.

After obtaining the above feature set, we conduct pre-training using ensemble learning algorithms like LightGBM. We perform feature selection by assessing the contribution of each feature to prediction accuracy (i.e., feature importance). Features ranked at the bottom of the importance list are removed.

TABLE I. PARAMETER SETTINGS FOR CASE GENERATION

Parameter	*Value range*	*Parameter*	*Value range*
S_m	[1,26]	A_n	[20,92]
P_m	[10,68]	E	[63,462]
C_m	[1,30]	F	[1,12]
U_{im}	[4,23]	D_{ij}	[0,6]
Q_{mn}	[1,2]	T_{ij}	[1,4]
G_i	[0,10]	$BigM$	1000
H_i	[10,39]	$\alpha_1, \alpha_2, \alpha_3$	1

On the above basis, the training of the supervised learning algorithm can be initiated. Rather than using time-consuming Support Vector Machines (SVMs), Logistic Regression (LR) with poor accuracy, and deep learning which may be difficult to tune [16] and with inadequate interpretability [17], we finally pick ensemble learning methods due to their satisfactory accuracy, acceptable training efficiency, manageable tuning complexity and qualified interpretability. Some experiments are conducted to select the most accurate ensemble learning algorithm among XGBOOST, Gradient Boosting Decision Tree (GBDT), Random Forest (RF), and LightGBM. Afterwards, cross-validation and hyperparameter optimization for further performance improvement.

TABLE II. COMPARISON ACROSS DIFFERENTIATED VALID CUT BUNDLE SELECTION WAYS

Cut Adding Setting	*Result*	*Case 1*	*Case 2*	*Case 2*	*Case 4*	*Case 5*
Differentially selecting valid cut bundles for the root node and non-root nodes	The optimal cut bundle	(0, 1, 1, 1, 1, 0)	(1, 1, 1, 1, 0, 0)	(1, 1, 0, 1, 1, 1)	(1, 1, 1, 1, 1, 0)	(1, 1, 1, 0, 1, 1)
	Computation time(s)	2.59	2.53	63.12	118.57	84.99
Adding valid cuts only at the root node	The optimal cut bundle	(0, 1, 1, 0, 0, 0)	(1, 1, 1, 0, 0, 0)	(1, 1, 1, 0, 0, 0)	(1, 1, 1, 0, 0, 0)	(1, 1, 1, 0, 0, 0)
	Computation time(s)	2.60	2.63	73.27	130.54	92.45
Adding valid cuts only at the non-root nodes	The optimal cut bundle	(0, 0, 0, 1, 1, 0)	(0, 0, 0, 1, 1, 1)	(0, 0, 0, 1, 0, 0)	(0, 0, 0, 0, 0, 1)	(0, 0, 0, 1, 0, 1)
	Computation time(s)	2.61	5.76	75.30	381.57	862.15
Globally same bundle application	The optimal cut bundle	(0, 0, 0, 0, 0, 0)	(1, 1, 1, 1, 1, 1)	(1, 1, 0, 1, 1, 0)	(1, 1, 1, 1, 1, 1)	(1, 1, 1, 1, 1, 1)
	Computation time(s)	2.60	2.56	65.62	118.89	90.02

IV. NUMERICAL EXPERIMENT

In this section, multiple cases are generated based on the settings in Table I. The numerical experiments first demonstrate the aforementioned claim that incorporating all valid cuts into the model does not necessarily yield the highest

computation efficiency, thereby highlighting the necessity of differentially selecting valid cut bundles across different stages in the branch-and-bound algorithm. Subsequently, we validate that the supervised-learning-based cut bundle selection method can effectively handle this issue while achieving high prediction accuracy.

A. Settings for Case Generation

Before presenting the numerical results, we first illustrate the settings for case generation. Each case corresponds to a MILP model's input parameters sampled from uniform distributions. The ranges of parameters are shown in Table I. These generated cases serve as the basis for the subsequent experiments.

It should be noted that the objective weights are manually assigned for easy demonstration. Similar to the settings in Jiang et al. [18] and Zhang et al. [6], we equally set the weights for different objectives. This is because that it is difficult to judge the actual importance of all objective terms, and they should be determined by particular decision-makers according to their personalized requirements. Meanwhile, manually setting the weights will not affect the mission of numerical experiments (e.g. to display our algorithm and validate the contributions).

B. The Necessity of Differentiated Selection across Different Solving Stages

The experiments numerically justify the differentiated valid cut bundle selection for root and non-root nodes in the branch-and-bound procedure by analyzing five randomly generated cases. The experimental framework compares the computation time under different settings of adding valid cuts.

As documented in Table II, the optimal bundle and corresponding computation time of each case are recorded. During the experiments, we employ a vector representation to describe the valid cut bundle, reflecting the selection of valid cuts at the root node and non-root nodes simultaneously. Therefore, the length of the vector should be twice the total number of valid cuts. As an example, the vector (0,0,1,1,0,0) indicates that the 3rd valid cut is applied at the root node, while the 1st valid cut is applied at the non-root nodes.

The experiment confirms that the optimal cut bundle differs across cases, and may not always be the entire candidate cut set. Furthermore, it reveals that differentially employing cut bundles on root-node stage and non-root node stage achieves a higher computation efficiency, than either adding cut bundles at only one type of node or applying identical bundles at both two types of nodes.

C. Supervised Learning Algorithms for the Differentiated Cut Bundle Selection

To analyze the prediction accuracy of supervised learning algorithms, we generate 4,928 samples by incorporating valid cut bundles for both root and non-root nodes during the branch-and-bound process for various cases. These samples are partitioned with 67% allocated for algorithm training, while the remaining is split into validation and testing sets. During preprocessing, we remove cases with infeasible solutions and eliminate outliers and missing values. Afterwards, by analyzing feature importance rankings from ensemble learning algorithms like LightGBM, we discard the last 30% of features to refine the feature set. To further improve the accuracy, we use the 10-fold cross-validation and grid search for hyperparameter tuning.

The conventional R^2 metric is employed to quantify the discrepancy between predicted and actual outcomes on the test set, serving as the basis for evaluating the prediction accuracy of supervised learning algorithms. Following a systematic comparison of mainstream supervised learning algorithms, GBDT is eventually selected due to its 99.5% prediction accuracy, which satisfies practical application requirements (see Table III for details). It should be noted that the conclusion that GBDT is the proper choice with such high accuracy may be sample-specific. If new samples are encountered in the future, the algorithm selection and validation process could follow a similar way to that described in this paper.

TABLE III. COMPARISON ACROSS POPULAR SUPERVISED LEARNING ALGORITHMS

Supervised learning algorithms	***XGBOOST***	***GBDT***	***RF***	***SVM***	***LR***	***LightGBM***
Prediction accuracy (%)	85.2	99.5	99.0	12.1	57.6	87.2

V. CONCLUSION AND DISCUSSION

In this paper, we present an integrated framework to optimize the configuration and delivery of the PSS by enhancing computation efficiency through the utilization of valid cut bundles. Specifically, we design customized valid cuts and introduce an ML algorithm for differentiated valid cut bundle selection in the branch-and-bound procedure. This approach not only considers the case-specific characteristics but also recognizes the requirement discrepancies across different solving stages. As one of the promising directions for future research, the current framework relies on hand-crafted statistical features, which may lead to primary information loss. It would be worthwhile to explore automated feature construction techniques using generative models (e.g., GPT) to enhance feature representation.

ACKNOWLEDGMENT

The authors would thank Maohua Cai for the technical and engineering support as well as the idea motivation of our research. We also express our gratitude to the reviewers and editors for their insightful comments, which have significantly improved the quality of this work.

REFERENCES

[1] Annarelli, A., C. Battistella, and F. Nonino, Product service system: A conceptual framework from a systematic review. Journal of Cleaner Production, 2016. 139: p. 1011-1032.

[2] Shen, J., et al., A framework for cost evaluation in product service system configuration. International Journal of Production Research, 2017. 55(20): p. 6120-6144.

[3] Machchhar, R.J., et al., Data-driven value creation in Smart Product-Service System design: State-of-the-art and research directions. Computers in Industry, 2022. 137: p. 103606.

[4] Zambetti, M., et al., From data to value: conceptualising data-driven product service system. Production Planning & Control, 2023. 34(2): p. 207-223.

[5] Zhang, Y., J. Chen, and Z. Jiang, Optimal product service system configuration considering pairing utility and uncertain customer behavior. Flexible Services and Manufacturing Journal, 2023. 35(2): p. 343-375.

[6] Zhang, Y., et al., Joint optimization of product service system configuration and delivery with learning-based valid cut selection and a tailored heuristic. Transportation Research Part E: Logistics and Transportation Review, 2024. 187: p. 103578.

[7] Cornuéjols, G., Valid inequalities for mixed integer linear programs. Mathematical Programming, 2008. 112(1): p. 3-44.

[8] Wolsey, L.A., Technical Note-Facets and Strong Valid Inequalities for Integer Programs. Oper. Res., 1976. 24(2): p. 367–372.

[9] Wei, X., et al., Tugboat scheduling for container ports. Transportation Research Part E: Logistics and Transportation Review, 2020. 142: p. 102071.

[10] You, J., Y. Wang, and Z. Xue, An exact algorithm for the multi-trip container drayage problem with truck platooning. Transportation Research Part E: Logistics and Transportation Review, 2023. 175: p. 103138.

[11] Huang, Z., et al., Learning to select cuts for efficient mixed-integer programming. Pattern Recognition, 2022. 123: p. 108353.

[12] Tang, Y., S. Agrawal, and Y. Faenza, Reinforcement Learning for Integer Programming: Learning to Cut, in Proceedings of the 37th International Conference on Machine Learning, D. Hal, III and S. Aarti, Editors. 2020, PMLR: Proceedings of Machine Learning Research. p. 9367--9376.

[13] Paulus, M.B., et al., Learning to Cut by Looking Ahead: Cutting Plane Selection via Imitation Learning, in Proceedings of the 39th International Conference on Machine Learning, C. Kamalika, et al., Editors. 2022, PMLR: Proceedings of Machine Learning Research. p. 17584--17600.

[14] Turner, M., et al., Adaptive Cut Selection in Mixed-Integer Linear Programming. Open Journal of Mathematical Optimization, 2023. 4: p. 1-28.

[15] Chicco, D., M.J. Warrens, and G. Jurman, *The coefficient of determination R-squared is more informative than SMAPE, MAE, MAPE, MSE and RMSE in regression analysis evaluation.* Peerj computer science, 2021. **7**.

[16] Martinović, M., K. Dokic, and D. Pudić, *Comparative Analysis of Machine Learning Models for Predicting Innovation Outcomes: An Applied AI Approach.* Applied Sciences, 2025. **15**(7): p. 3636.

[17] Mendonça, M.O.K., et al., *Chapter 13 - Machine learning: Review and trends*, in *Signal Processing and Machine Learning Theory*, P.S.R. Diniz, Editor. 2024, Academic Press. p. 869-959.

[18] Jiang, S., et al., Data-Driven Optimization for Dynamic Shortest Path Problem Considering Traffic Safety. IEEE Transactions on Intelligent Transportation Systems, 2022. 23(10): p. 18237-18252.